



Figure 4: Testing the *dev-user*

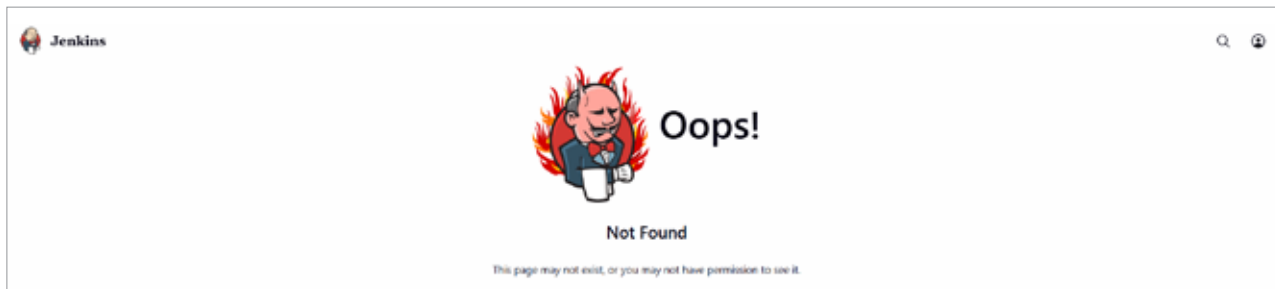


Figure 5: Testing by link (*dev-user*)



Figure 6: Testing by link (*ops-team*)

in action. Testing is the most important step to ensure you haven't accidentally locked everyone out or left a security hole open.

Log in as the *ops-team* user. Because we used the `*.*` Regex pattern, this user should see every job on the dashboard, including *Alpha-Project* and *Beta-Project*. This confirms that your administrative and oversight roles are working correctly as shown in Figure 3.

Now, log in as the *dev-user*. If your regex `Alpha-.*` is set up correctly, this user will only see jobs starting with the 'Alpha-' prefix. The *Beta-Project* will be completely invisible to them, proving that the environment is now isolated as shown in Figure 4.

A final professional test is to try and 'cheat' the system. While logged in as *dev-user*, try to manually type the

URL of a Beta job into your browser (e.g., `localhost:8081/job/Beta-Project`). Jenkins should instantly display an *Access Denied* error. This proves that your security is enforced at the server level, not just the visual UI (Figure 5).

The same link (`localhost:8081/job/Beta-Project`), while logged in as *ops-team*, displays the information related to the project as shown in Figure 6.

Security should never be an afterthought. With these settings, your Jenkins server is no longer just a build tool — it is a hardened, production-ready automation platform.

Hardening a Jenkins server requires ongoing effort, but implementing

Role-Based Access Control is the most critical step towards a professional and secure CI/CD environment. By moving away from 'Admin-for-Everyone' and utilising the *Role-Based Authorization Strategy* plugin, you immediately reduce risk by preventing users from accidentally deleting unauthorised jobs, improve focus by letting developers only see relevant projects, and achieve enterprise readiness, allowing the system to scale securely to hundreds of users and projects while protecting sensitive data, thus transforming the Jenkins server from a mere build tool into a hardened, production-ready automation platform. **END** 🐧

By: Arun Santhosh R.A.

The author is a FOSS enthusiast with an interest in data security and backup.